🎯 20 JavaScript Interview Questions Cheat Sheet

A cheat sheet covering 20 common JavaScript interview questions with answers and code demos.

> 💡 **Tip**
>
> Complete Dev Roadmap with SaaS Boilerplate at thedevspace.io/

1. How do you reverse a string?

Split into characters, reverse the array, then join back into a string.

```
const reversed = str.split("").reverse().join("");
```

2. How do you check for a palindrome?

Compare the string to its reversed form to test symmetry.

```
function isPalindrome(s) {
  return s === s.split("").reverse().join("");
}
```

3. How do you find the factorial of a number?

Use recursion or an iterative loop for small inputs.

```
function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
}
```

4. How do you find the largest/smallest number in an array?

Use Math.max or Math.min with the spread operator for plain arrays.

```
const max = Math.max(...arr);
const min = Math.min(...arr);
```

5. How do you remove duplicates from an array?

Convert to a Set and back to an array to remove duplicates quickly.

```
const unique = [...new Set(arr)];
```

6. How do you flatten a nested array?

Use Array.flat for shallow or deep flattening with Infinity for full depth.

```
const flat = arr.flat(Infinity);
```

7. How do you find the intersection of two arrays?

Filter one array by membership in the other for a simple intersection.

```
const intersection = a.filter((x) => b.includes(x));
```

8. How do you merge two sorted arrays?

Use two pointers to merge in linear time without extra sorting.

```
function merge(a, b) {
  const out = [];
  let i = 0,
    j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] < b[j]) out.push(a[i++]);
    else out.push(b[j++]);
  }
  return out.concat(a.slice(i), b.slice(j));
}
```

9. How do you implement binary search?

Use a loop that halves the search range on each step for a sorted array.

```
function binarySearch(arr, x) {
  let l = 0,
    r = arr.length - 1;
  while (l <= r) {
    const m = Math.floor((l + r) / 2);
    if (arr[m] === x) return m;
    if (arr[m] < x) l = m + 1;
    else r = m - 1;
  }
  return -1;
}
```

10. How do you implement bubble sort?

Compare adjacent elements and swap to bubble largest items to the end.

```
function bubbleSort(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length - i - 1; j++) {
      if (arr[j] > arr[j + 1]) [arr[j], arr[j + 1]]
= [arr[j + 1], arr[j]];
    }
  }
  return arr;
}
```

11. How do you implement quick sort?

Pick a pivot, partition elements, then recursively sort partitions.

```
function quickSort(a) {
  if (a.length <= 1) return a;
  const p = a[Math.floor(a.length / 2)];
  const left = a.filter((x) => x < p);
  const mid = a.filter((x) => x === p);
  const right = a.filter((x) => x > p);
  return [...quickSort(left), ...mid,
...quickSort(right)];
}
```

12. How do you implement merge sort?

Split the array, sort halves recursively, then merge them.

```
function mergeSort(a) {
  if (a.length <= 1) return a;
  const mid = Math.floor(a.length / 2);
  const left = mergeSort(a.slice(0, mid));
  const right = mergeSort(a.slice(mid));
  return merge(left, right);
}
```

13. How do you find the nth Fibonacci number?

Use iteration for efficiency or recursion for simplicity on small n.

```
function fib(n) {
  let a = 0,
    b = 1;
  for (let i = 0; i < n; i++) {
    [a, b] = [b, a + b];
  }
  return a;
}
```

14. How do you check if two strings are anagrams?

Sort and compare or count characters to compare frequency maps.

```
function isAnagram(a, b) {
  return a.split("").sort().join("") ===
b.split("").sort().join("");
}
```

15. How do you count the number of vowels in a string?

Match vowels with a regex and count matches, or loop and tally.

```
const count = (s.match(/[aeiou]/gi) || []).length;
```

16. How do you find the first non-repeating character?

Count occurrences then return the first character with a single count.

```
function firstUnique(s) {
  const m = new Map();
  for (const c of s) m.set(c, (m.get(c) || 0) + 1);
  for (const c of s) if (m.get(c) === 1) return c;
  return null;
}
```

17. How do you implement a stack?

Use an array with push and pop to get LIFO behavior.

```
const stack = [];
stack.push(1);
stack.pop();
```

18. How do you implement a queue?

Use an array with push and shift for FIFO, or a linked list for heavy workloads.

```
const q = [];
q.push(1);
q.shift();
```

19. How do you detect a cycle in a linked list?

Use two pointers moving at different speeds (Floyd's algorithm) to detect loops.

```
function hasCycle(head) {
  let slow = head,
    fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

20. How do you traverse a binary tree?

Use recursion for in-order, pre-order, or post-order traversal.

```
function inOrder(node, visit) {
  if (!node) return;
  inOrder(node.left, visit);
  visit(node.value);
  inOrder(node.right, visit);
}
```